TECHNICAL DESIGN DOCUMENT

Grid-Guard AI

System & Database Design

Tech stack, database schema, repo structure, and deployment plan for the web dashboard and mobile app

Field	Value
Author	Noor Eman
Version	1.0 (Draft)
Related documents	Grid-Guard AI PRD; Architecture Document; API Specification
Date	August 2026

1. Purpose

This document translates the PRD's functional requirements into concrete engineering decisions: what technologies are used, how data is modeled, how the codebase is organized, and how the system is deployed. It answers “how do we build this” so that implementation can start without ad-hoc decisions being made mid-development.

2. Technology Stack

Layer	Choice	Why
ML inference	Python, scikit-learn/XGBoost model loaded via joblib, served through the backend	Matches the model trained per the ML Strategy document; no separate ML service needed at this scale
Backend API	FastAPI (Python)	Same language as the ML pipeline — the trained model can be loaded directly in-process, no cross-language serialization needed
Database	PostgreSQL	Free-tier hosting available (Render/Supabase), relational structure fits the case/consumer/inspection model well
Object storage (photos)	Local filesystem for MVP → S3-compatible free tier (e.g. Cloudflare R2 free tier) for V1	Avoids over-engineering storage before it's needed; easy to swap later since it's abstracted behind one upload function
Web frontend	React + Next.js	Widely documented, fast to build dashboards with, easy free deployment on Vercel
Mobile app	Flutter	Single codebase, strong offline/local-storage support (sqflite package), good free-tier friendliness (no per-platform native code needed)
Mobile local storage	SQLite (via sqflite)	Enables the offline-first requirement (MOB-5) — cases and pending inspections persist locally until synced
Auth	JWT (JSON Web Tokens)	Stateless, works cleanly across both web and mobile clients without server-side session storage

3. Database Schema

Six core tables. Relationships are kept simple and normalized — no premature denormalization, since data volume at this project's scale (thousands, not millions, of consumers) does not require it.

3.1 users

Covers both officer and inspector roles, distinguished by the role column, rather than separate tables — they share the same auth mechanism.

Column	Type	Notes
id	UUID, PK	
name	text	
role	enum('officer','inspector','circle_manager')	Drives which UI the client shows and which endpoints are permitted
email	text, unique	Login identifier
password_hash	text	bcrypt hash, never store plaintext
created_at	timestamp	

3.2 consumers

Represents the benchmark/synthetic consumer records described in the ML Strategy document — not live DISCO data. A is_demo_data flag is included so the UI can clearly label this, consistent with the PRD's data-honesty requirement.

Column	Type	Notes
id	UUID, PK	
external_ref	text	Consumer/meter ID from the source dataset (SGCC/PRECON)
address	text	
feeder	text	Used for WEB-1 filtering and the feeder map (WEB-6)
tariff_class	text	
meter_type	enum('conventional','ami')	Reflects the mixed-metering reality established in the proposal
is_demo_data	boolean, default true	Drives the “demo data” UI label required by the PRD

3.3 risk_scores

One row per scoring run per consumer — kept as a history table (not overwritten) so the Model Performance Panel (WEB-7) can show trends over time.

Column	Type	Notes
id	UUID, PK	
consumer_id	UUID, FK → consumers.id	
score	float (0–1)	
risk_band	enum('high','medium','low')	Derived from score using thresholds defined in the ML output schema (see Section 4)
explanation_text	text	Plain-language SHAP explanation, precomputed at scoring time (WEB-2, MOB-2)

Column	Type	Notes
explanation_raw	jsonb	Raw SHAP feature/value pairs, kept for audit/debugging, not shown directly in UI
scored_at	timestamp	

3.4 cases

Column	Type	Notes
id	UUID, PK	
consumer_id	UUID, FK → consumers.id	
risk_score_id	UUID, FK → risk_scores.id	Which scoring run triggered this case
assigned_by	UUID, FK → users.id	Officer who assigned it
assigned_to	UUID, FK → users.id	Inspector
status	enum('assigned','in_progress','closed_confirmed','closed_false_positive','closed_inconclusive')	
assignment_note	text	Optional note from officer (WEB-3)
created_at / updated_at	timestamp	

3.5 inspections

Column	Type	Notes
id	UUID, PK	
case_id	UUID, FK → cases.id	
inspector_id	UUID, FK → users.id	
outcome	enum('confirmed','false_positive','inconclusive')	
tampering_type	text[]	Checklist selections (MOB-3)
note	text	
photo_url	text	Path/URL to stored photo
gps_lat / gps_lng	float	Auto-captured at submission time (MOB-4)

Column	Type	Notes
submitted_at	timestamp	Time of submission on-device — may differ from synced_at
synced_at	timestamp, nullable	Set when the record reaches the server; null while pending sync

3.6 audit_log

Append-only. Every state-changing action across the system writes one row here, satisfying WEB-5. Never updated or deleted.

Column	Type	Notes
id	UUID, PK	
actor_id	UUID, FK → users.id, nullable	Null for system-generated events (e.g. model flagging)
action	text	e.g. 'case_assigned', 'inspection_submitted', 'flag_generated'
entity_type / entity_id	text / UUID	What the action was performed on
details	jsonb	Free-form context for the action
created_at	timestamp	

4. ML Output Schema (Contract with the Backend)

The trained model (joblib file, per the ML Strategy document) must be wrapped by a single scoring function returning a fixed shape, so the backend never depends on model internals:

```
{
  "consumer_id": "uuid",
  "score": 0.91,
  "risk_band": "high",
  "explanation_text": "Consumption dropped 60% ...",
  "explanation_raw": { "feature": "consumption_drop_3mo", "shap_value": 0.42, ...
}
}
```

Risk band thresholds (adjustable, defined once in one config location, not hardcoded in multiple places): score ≥ 0.7 → high; 0.4–0.69 → medium; below 0.4 → low.

5. Repository Structure

A monorepo is recommended for a solo builder — avoids the overhead of coordinating three separate repos and their versioning for a project without a multi-team release process.

```
grid-guard-ai/
├── backend/
│   ├── app/
│   │   ├── models/        (SQLAlchemy models mirroring Section 3)
│   │   ├── routers/        (one file per resource: consumers, cases,
inspections, auth)
│   │   ├── ml/              (model loading + scoring wrapper, Section 4
```

```
contract)
│   │   ├── auth/           (JWT issuing/verification)
│   │   └── main.py
│   ├── ml_artifacts/
│   │   └── model.pkl        (trained model from the ML pipeline)
│   └── requirements.txt
├── web/                        (Next.js app)
│   └── src/
│       ├── pages/            (one per screen — see UI/UX Flow document)
│       └── components/
├── mobile/                     (Flutter app)
│   └── lib/
│       ├── screens/
│       ├── local_db/        (sqflite schema + sync queue)
│       └── api/
└── docs/                       (this document, PRD, API spec, architecture doc)
```

6. Offline Sync Strategy (Mobile)

Directly implements MOB-5. The mobile app maintains a local SQLite table mirroring a subset of inspections, with a synced boolean flag.

1. On case fetch (when online), assigned cases are cached locally in SQLite.
2. Inspection submissions are always written to the local SQLite table first, regardless of connectivity.
3. A background sync process checks connectivity periodically (and on app foreground); for each unsynced row, it POSTs to /cases/{id}/inspection.
4. On success, the local row is marked synced=true and synced_at is set; on failure, it remains queued and retries on the next check.
5. Conflict policy: last-write-wins at the case level — acceptable at this project's scale since a case is assigned to exactly one inspector, so concurrent conflicting writes are not expected.

7. Deployment Plan

Component	Hosting (free tier)	Notes
Backend API + DB	Render (web service) + Render PostgreSQL, or Supabase	Both offer free tiers sufficient for a portfolio project's traffic level
Web frontend	Vercel	Native Next.js support, generous free tier
Mobile app	Distributed as an APK / TestFlight build for demo purposes	Full app-store listing not required for a portfolio project
Photo storage	Cloudflare R2 free tier (10GB)	S3-compatible; swappable without changing application code if abstracted behind one upload interface

8. Configuration & Secrets

- All secrets (DB connection string, JWT signing key, storage credentials) via environment variables, never committed to the repo.

- A .env.example file checked into the repo documents required variables without exposing real values.
- Risk-band thresholds (Section 4) and other tunable constants live in one config module, not scattered across route handlers.